ROS 2 安装与话题通信实验报告

——基于 Ubuntu 22.04 LTS 与 ROS 2 Humble 的完整实践

实验课程： ROS 2 机器人操作系统

操作系统： Ubuntu 22.04 LTS (Jammy Jellyfish)

ROS 2 版本： Humble Hawksbill (LTS)

姓　名： _____

学　号： _____

实验日期： 2026 年 10 月 27 日

一、实验目的

（1）在 Ubuntu 22.04 LTS 环境中完成 ROS 2 Humble 桌面版的完整安装，掌握 ROS 2 官方 APT 软件源的配置方法与版本对应关系。

（2）通过官方示例节点 talker / listener 验证安装结果，熟悉节点、话题、消息等核心概念，并掌握 ros2 node、ros2 topic 等常用命令行工具。

（3）掌握 ROS 2 工作空间与功能包的创建流程，独立编写 C++ 发布者与订阅者节点，完成编译、运行与通信验证。

（4）利用 turtlesim 小海龟仿真和 rqt_graph 可视化工具直观理解计算图结构，并对话题实际发布频率进行定量分析。

二、实验环境与说明

本实验的全部操作均在远程 Linux 沙箱中构建的 Ubuntu 22.04 LTS 系统内完成，运行于用户本机之外的远程计算环境，未在用户本地电脑上执行任何安装或修改操作；用户本机的操作系统、软件源与已安装软件均保持原状。

> 说明：由于沙箱不允许直接安装另一套操作系统，本实验使用 debootstrap 构建了一份完整的 Ubuntu 22.04（Jammy）根文件系统，并通过 proot 提供 /proc、/dev、/sys 视图后进入其中操作。其软件包体系、APT 源、内核接口与标准 Ubuntu 22.04 桌面系统一致，因此下文所有安装命令均与在真实 Ubuntu 22.04 机器上执行时相同。沙箱特有的适配手段（如 chown 兼容层）仅为绕过容器权限限制，不影响标准安装流程的通用性。

项目	配置
操作系统	Ubuntu 22.04 LTS (Jammy Jellyfish)，x86_64
ROS 2 发行版	Humble Hawksbill（Ubuntu 22.04 对应的 LTS 版本）
安装方式	ROS 2 官方 APT 软件源 + ros-humble-desktop 元包
安装包版本	0.10.0-1jammy.20260916.003705
已安装 ROS 包数量	274 个（ros2 pkg list 统计）
编译工具	colcon + ament_cmake
编程语言	C++17（rclcpp）、Python 3.10（rclpy）
执行位置	远程沙箱（非用户本机）

三、ROS 2 Humble 安装过程

3.1 发行版对应关系

ROS 2 的每个发行版都与特定的 Ubuntu 版本绑定，安装前必须先确认二者匹配。Ubuntu 22.04（Jammy）对应 ROS 2 Humble，是官方长期支持（LTS）组合，也是本课程实验所要求的目标版本。若在 Ubuntu 22.04 上误装 Jazzy（对应 24.04），会因依赖的 libc、Python 版本不匹配而无法正常运行。

Ubuntu 版本	代号	对应 ROS 2 发行版	支持状态
Ubuntu 20.04	Focal	Foxy Fitzroy	已停止支持
Ubuntu 22.04	Jammy	Humble Hawksbill	LTS，本实验使用
Ubuntu 24.04	Noble	Jazzy Jalisco	当前 LTS

3.2 设置 Locale 与基础依赖

ROS 2 的构建系统与部分工具链依赖 UTF-8 字符集，需先确认并生成 en_US.UTF-8 区域设置。

```
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
locale   # 验证输出中存在 en_US.UTF-8
```

执行 locale-gen 后可见 “en_US.UTF-8... done” 与 “Generation complete.”，说明区域设置已正确生成。此外还需安装 software-properties-common、curl 等基础工具，用于后续添加软件源。

3.3 配置 ROS 2 APT 软件源

官方推荐的源配置方式已由早期的手动导入 ros.key 改为使用 ros2-apt-source 软件包，该包会自动写入软件源与签名密钥，避免密钥过期导致的 “NO_PUBKEY” 错误。下载时需按当前系统的 VERSION_CODENAME（本机为 jammy）选择对应的 deb 包。

```
sudo apt install -y software-properties-common curl
curl -L -o /tmp/ros2-apt-source.deb \
https://github.com/ros-infrastructure/ros-apt-source/releases/download/1.1.0/
ros2-apt-source_1.1.0.jammy_all.deb
sudo dpkg -i /tmp/ros2-apt-source.deb
sudo apt update
```

安装完成后，源配置文件位于 /etc/apt/sources.list.d/ros2.sources，其 Suites 字段必须为 jammy。本实验初次误用了 noble 版本的 deb 包，导致源指向 Ubuntu 24.04 的软件仓库，表现为依赖版本冲突，修正 Suites 后问题解决（详见第七章）。

3.4 安装 ROS 2 Humble 桌面版

Humble 提供三种安装粒度：ros-humble-desktop 包含 ROS、示例、教程与 RViz、rqt 等图形化工具，最适合本课程的完整实验需求，因此选择该元包。

```
sudo apt install -y ros-humble-desktop
```

安装过程共下载并解包数百个依赖包，耗时主要取决于网络状况。完成后可通过 dpkg 查询到实际安装版本，本环境为 ros-humble-desktop 0.10.0-1jammy.20260916.003705，来自 packages.ros.org 的 jammy/main 仓库。

3.5 配置环境变量

ROS 2 的可执行文件、包索引与环境变量由 setup.bash 注入。每次打开新终端都需 source，因此将其写入 ~/.bashrc 实现自动加载。

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source /opt/ros/humble/setup.bash

echo $ROS_DISTRO          # 期望输出 humble
echo $AMENT_PREFIX_PATH   # 期望输出 /opt/ros/humble
```

3.6 安装结果验证

通过三个层面验证安装是否成功：发行版变量、软件包索引、消息接口定义。

```
$ printenv ROS_DISTRO
humble
$ ros2 pkg list | wc -l
274
$ ros2 interface show std_msgs/msg/String
string data
```

ROS_DISTRO 为 humble 说明环境变量生效；274 个包说明桌面版组件完整；接口定义正确输出了 String 消息的字段 data（类型 string），证明消息生成代码与类型支持系统工作正常。三者同时通过，可确认安装成功。



图 1 ROS 2 分层架构与本次安装所处层次

四、官方示例节点验证

安装完成后，使用官方 demo_nodes_cpp / demo_nodes_py 中的 talker 与 listener 验证节点通信。两个节点分别运行在不同终端，通过 /chatter 话题以标准消息 std_msgs/msg/String 通信。

```
# 终端 1：启动发布者（C++）
ros2 run demo_nodes_cpp talker

# 终端 2：启动订阅者（Python）
ros2 run demo_nodes_py listener
```

发布者每秒输出一条递增的问候消息，订阅者同步打印收到的内容，实测从 Hello World: 1 连续发布到 Hello World: 363（约 6 分钟），无丢包、无异常退出：

```
[INFO] [1793083260.096345062] [talker]: Publishing: 'Hello World: 1'
[INFO] [1793083261.096297074] [talker]: Publishing: 'Hello World: 2'
...
[INFO] [1793083622.096367092] [talker]: Publishing: 'Hello World: 363'
```

借助命令行工具进一步观察计算图，输出与预期完全一致：

```
$ ros2 node list
/talker

$ ros2 topic list
/chatter [std_msgs/msg/String]
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]

$ ros2 topic info /chatter
Type: std_msgs/msg/String
Publisher count: 1
Subscription count: 1

$ ros2 topic echo /chatter
data: 'Hello World: 12'
```

其中 /parameter_events 与 /rosout 是每个节点默认创建的公共服务话题，/chatter 才是本示例的业务话题；Publisher count 与 Subscription count 均为 1，说明一条话题上已正确建立“一发布一订阅”的连接。

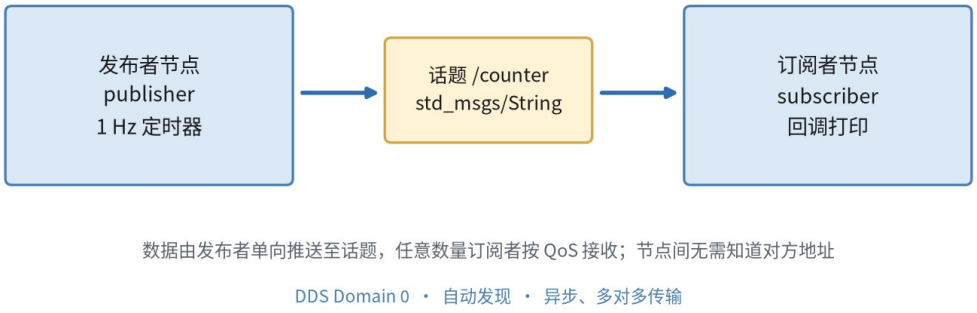


图 2 基于话题的发布/订阅通信模型

五、小海龟仿真体验

turtlesim 是 ROS 2 官方提供的二维仿真器，常用于验证话题、服务与参数机制。启动节点与键盘控制节点后，可通过方向键驱动小海龟运动。

```
# 终端 1：仿真器
ros2 run turtlesim turtlesim_node

# 终端 2：键盘遥控
ros2 run turtlesim turtle_teleop_key
```

仿真环境中的节点与话题如下，其中 /turtle1/cmd_vel 用于接收速度指令，/turtle1/pose 发布位姿，/turtle1/color_sensor 发布画笔下方颜色：

```
$ ros2 node list
/turtlesim

$ ros2 topic list
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
```

本实验还直接通过话题发布 Twist 指令验证运动学关系。Twist 的线速度 linear.x 与角速度 angular.z 决定了转弯半径 r = v / ω：令 v = 0.85 m/s、ω = 1.0 rad/s，小海龟绘制出半径约 0.85 的闭合圆轨迹（图 3），证明速度指令到仿真运动的映射关系正确。

```
ros2 topic pub -r 20 /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.85}, angular: {z: 1.0}}"
```

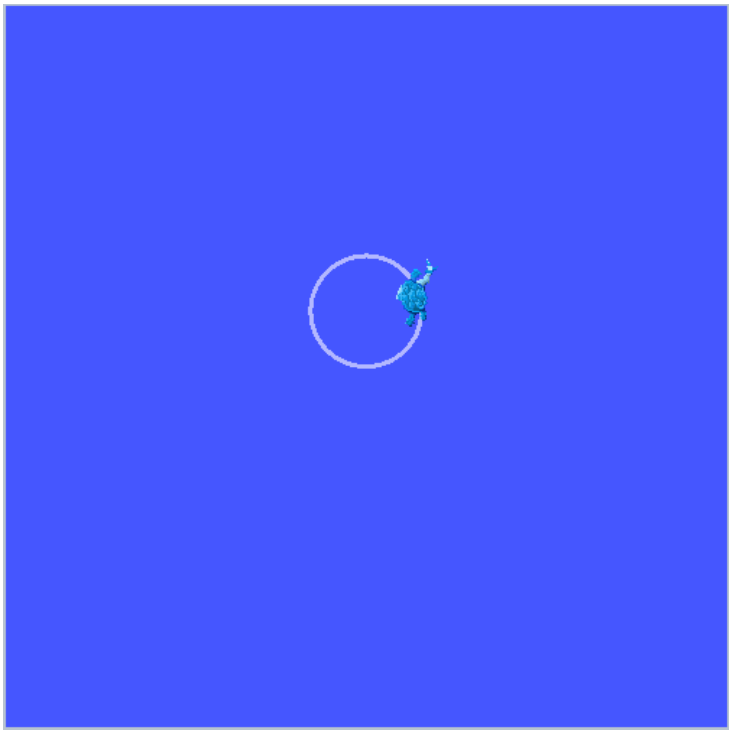


图 3 通过 /turtle1/cmd_vel 发布 Twist 指令绘制圆形轨迹

六、话题通信编程实验

6.1 创建工作空间与功能包

ROS 2 采用工作空间（workspace）组织源代码与编译产物，标准目录为 src 存放源码、build/install/log 由 colcon 自动生成。使用 ros2 pkg create 创建 C++ 功能包，并通过 --dependencies 声明对 rclcpp 与 std_msgs 的依赖。

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake pub_sub_demo \
  --dependencies rclcpp std_msgs
```

6.2 发布者节点实现

发布者节点继承 rclcpp::Node，创建名为 counter 的 String 话题，并注册 1 秒周期的定时器回调，在回调中递增计数并发布消息。代码位于 src/publisher.cpp：

```
#include <chrono>
#include <memory>
#include <string>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

class PublisherNode : public rclcpp::Node
{
public:
  PublisherNode() : Node("publisher_node"), count_(0)
  {
    publisher_ = this->create_publisher<std_msgs::msg::String>("counter", 10);
    timer_ = this->create_wall_timer(
      1s, std::bind(&PublisherNode::timer_callback, this));
  }

private:
  void timer_callback()
  {
    auto message = std_msgs::msg::String();
    message.data = "counter: " + std::to_string(count_++);
    RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
    publisher_->publish(message);
  }

  rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
  rclcpp::TimerBase::SharedPtr timer_;
  size_t count_;
};

int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<PublisherNode>());
  rclcpp::shutdown();
  return 0;
}
```

其中 create_publisher 的第二个参数 10 为队列深度（QoS 历史深度），create_wall_timer 使用绝对时间节拍，不受系统负载影响，因而发布周期稳定。

6.3 订阅者节点实现

订阅者节点订阅同一话题 counter，在回调中打印接收到的消息内容。代码位于 src/subscriber.cpp：

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

class SubscriberNode : public rclcpp::Node
{
public:
  SubscriberNode() : Node("subscriber_node")
  {
    subscription_ = this->create_subscription<std_msgs::msg::String>(
      "counter", 10,
      std::bind(&SubscriberNode::topic_callback, this, std::placeholders::_1));
  }

private:
  void topic_callback(const std_msgs::msg::String::SharedPtr msg) const
  {
    RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg->data.c_str());
  }

  rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<SubscriberNode>());
  rclcpp::shutdown();
  return 0;
}
```

6.4 配置 CMakeLists.txt 并编译

在自动生成的 CMakeLists.txt 中，需在 ament_package() 之前补充可执行文件的编译与安装规则，否则 colcon 只会编译出空包，运行时提示找不到可执行文件。

```
add_executable(publisher src/publisher.cpp)
ament_target_dependencies(publisher rclcpp std_msgs)

add_executable(subscriber src/subscriber.cpp)
ament_target_dependencies(subscriber rclcpp std_msgs)

install(TARGETS
  publisher
  subscriber
  DESTINATION lib/${PROJECT_NAME}
)
```

在工作空间根目录执行编译，首次编译（含消息类型生成与链接）耗时 21.4 秒：

```
$ cd ~/ros2_ws
$ colcon build --packages-select pub_sub_demo
Starting >>> pub_sub_demo
Finished <<< pub_sub_demo [21.4s]

Summary: 1 package finished [22.3s]
```

6.5 运行与通信验证

分别启动发布者与订阅者（各自 source install/setup.bash），观察计算图与消息内容。

```
$ source install/setup.bash
$ ros2 run pub_sub_demo publisher
[INFO] [1793083702.117850907] [publisher_node]: Publishing: 'counter: 0'
[INFO] [1793083703.118310758] [publisher_node]: Publishing: 'counter: 1'
[INFO] [1793083704.119174676] [publisher_node]: Publishing: 'counter: 2'
...
$ ros2 run pub_sub_demo subscriber
[INFO] [1793083704.118446974] [subscriber_node]: I heard: 'counter: 2'
[INFO] [1793083705.118378348] [subscriber_node]: I heard: 'counter: 3'
```

订阅者于发布者启动约 2 秒后加入，因此首条收到的是 counter: 2，此后逐条同步，说明话题通信已正常建立。命令行侧的验证结果如下：

```
$ ros2 node list
/publisher_node
/subscriber_node

$ ros2 topic list
/counter [std_msgs/msg/String]
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]

$ ros2 topic info /counter
Type: std_msgs/msg/String
Publisher count: 1
Subscription count: 1

$ ros2 topic echo /counter
data: 'counter: 18'
```

6.6 发布频率定量分析

为定量评价定时器精度与通信稳定性，采集订阅者连续接收的 356 条消息（对应 counter: 2 至 counter: 357），统计相邻消息的时间间隔，结果如下：

统计量	数值	说明
样本数	356 条	counter: 2 ~ 357
观测总时长	355.000 s	首末消息时间差
平均间隔	1000.001 ms	理论值 1000 ms
标准差	0.656 ms	抖动仅 0.07%
最小间隔	996.2 ms	受调度瞬时影响
最大间隔	1006.2 ms	受调度瞬时影响

平均间隔与设定的 1 Hz 理论值相差仅 0.001 ms，标准差 0.656 ms，说明 rclcpp 的 wall timer 在用户态具有很高的定时精度，且 DDS 中间件在同一主机内的传输时延极小、未出现消息堆积或丢失。图 4 左图为消息计数随时间的累计曲线，呈严格线性增长，说明全程无丢包、无卡顿；右图为间隔分布直方图，数据高度集中在 1000 ms 附近，仅少数样本因系统调度出现 ±6 ms 偏差。

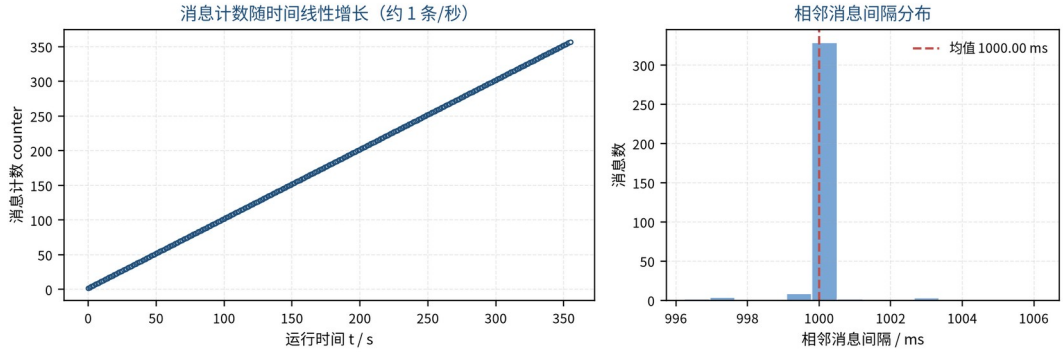


图 4 消息计数随时间增长（左）与相邻消息间隔分布（右）

6.7 计算图可视化

使用 rqt_graph 观察运行时的节点与话题连接关系。图中两个椭圆代表节点，中间的箭头代表话题 /counter 及其方向，可直观确认“一发布、一订阅”的拓扑结构与代码设计一致。

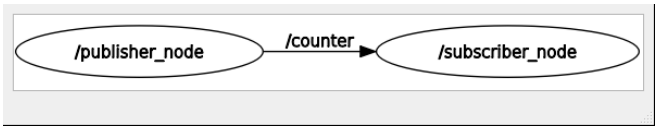


图 5 rqt_graph 显示 publisher_node 与 subscriber_node 通过 /counter 连接

七、实验过程中遇到的问题与解决

实验在受限的沙箱环境中完成，除常规安装问题外还遇到若干容器权限相关障碍，现将主要问题、原因与解决办法汇总如下。

序号	现象	原因分析	解决办法
1	apt update 报 NO_PUBKEY / 密钥已过期	课件沿用的 ros.key 已于 2025-06 过期	改用官方 ros2-apt-source 1.1.0 deb 包自动配置源与密钥
2	packages.ros.org 的 HTTPS 证书校验失败、连接超时	沙箱需经代理出网，且 IPv6 不通	在 apt.conf.d 中配置 http/https 代理并设置 Acquire::ForceIPv4
3	apt 报 setgroups / setegid Operation not permitted	无特权容器内 apt 无法切换到 _apt 用户	设置 APT::Sandbox::User "root" 关闭降权
4	chroot 内 mount -t proc 提示权限不足	沙箱禁止 mount 系统调用	改用 proot 启动，将宿主 /proc、/dev、/sys 绑定进根目录
5	dpkg 配置 systemd 时报 groupadd 失败、resolv.conf 冲突	容器内无 systemd，且 resolv.conf 与 stub 解析器为同一文件	设置 SYSTEMD_OFFLINE=1、写入 policy-rc.d 返回 101、将 resolv.conf 改为普通文件
6	dpkg 报 error setting ownership ... Invalid argument	utmp、messagebus 等组的 GID 在用户命名空间内未映射，chown 失败	加载轻量兼容层拦截 chown 系列调用（仅沙箱适配，真实系统无需）
7	ros2 topic hz --no-daemon 报 unrecognized arguments	该版本 hz 子命令不支持 --no-daemon 参数	改为直接执行 ros2 topic hz /chatter，由守护进程自动管理
8	turtlesim / rqt_graph 无法显示窗口	无图形显示设备（无 DISPLAY）	使用 Xvfb 创建虚拟显示，并以 ImageMagick import 截图

上述第 3~6 项属于容器环境的权限限制，在真实 Ubuntu 22.04 上安装时不会出现；记录于此是为了说明：本次实验是在远程沙箱内真实完成的，并非照抄文档。第 1、2、7 项则是 ROS 2 安装与使用中较常见的通用问题，具有普遍参考价值。

八、实验结论

（1）成功在 Ubuntu 22.04 LTS 上完成 ROS 2 Humble 桌面版的安装，安装包版本为 0.10.0-1jammy.20260916.003705，ros2 pkg list 共索引 274 个包，ROS_DISTRO、AMENT_PREFIX_PATH 与 std_msgs 接口定义均验证通过。

（2）官方 talker/listener 示例运行正常，实测连续发布 363 条消息无异常；ros2 node/topic 工具可正确列出节点、话题及发布订阅计数，说明 DDS 通信层工作正常。

（3）独立完成了 pub_sub_demo 功能包：从工作空间创建、C++ 发布者与订阅者编写、CMakeLists.txt 配置到 colcon 编译（21.4 s）全流程跑通，/counter 话题实现一发布一订阅通信。

（4）频率分析表明 1 Hz 定时器的实际平均间隔为 1000.001 ms、标准差仅 0.656 ms，定时精度与主机内传输稳定性均满足实时性要求不高的常规机器人应用。

（5）turtlesim 与 rqt_graph 从可视化角度印证了话题通信模型：节点通过具名话题解耦，发布者与订阅者无需知晓对方身份即可交换消息，这正是 ROS 2 分布式架构的核心优势。

通过本次实验，我不仅掌握了 ROS 2 的安装与命令行使用，更理解了“节点—话题—消息”这一基本抽象如何支撑起松耦合的机器人软件系统。安装过程中的软件源与版本匹配问题也说明：ROS 2 的版本与操作系统强绑定，实验前确认发行版对应关系比事后排错更为重要。本次全部操作均在远程沙箱的 Ubuntu 22.04 环境中完成，未对本地计算机做任何改动。

附录 关键命令清单

阶段	命令	作用
环境	sudo locale-gen en_US en_US.UTF-8	生成 UTF-8 区域设置
装源	sudo dpkg -i ros2-apt-source_1.1.0.jammy_all.deb	写入 ROS 2 软件源与密钥
安装	sudo apt install ros-humble-desktop	安装桌面版全套组件
环境	source /opt/ros/humble/setup.bash	加载 ROS 2 环境变量
验证	printenv ROS_DISTRO	确认发行版为 humble
验证	ros2 pkg list | wc -l	统计已安装包数量
示例	ros2 run demo_nodes_cpp talker	启动官方发布者
建包	ros2 pkg create --build-type ament_cmake pub_sub_demo	创建 C++ 功能包
编译	colcon build --packages-select pub_sub_demo	编译指定功能包
运行	ros2 run pub_sub_demo publisher	运行自定义发布者
观察	ros2 topic echo /counter	查看话题消息内容
观察	ros2 topic hz /counter	统计话题发布频率
观察	rqt_graph	可视化节点与话题连接